

Justificación del formato de almacenamiento

Para el almacenamiento de la configuración, se seleccionó JSON como el formato definitivo tras evaluar alternativas con base en criterios técnicos de robustez, facilidad de recorrido y manejo de tipos de datos. JSON se maneja directamente a diccionarios en Python y preserva los tipos nativos (como enteros para el tamaño de fuente), lo que evita conversiones manuales propensas a errores, además de permitir un manejo seguro de caracteres especiales mediante codificación UTF-8 y facilitar la detección de corrupción estructural a través de excepciones específicas. Por el contrario, se descartó el uso de formatos binarios; aunque ofrecen un tamaño de archivo mínimo y velocidad de lectura, sacrifican por completo la legibilidad humana, impidiendo que el usuario pueda inspeccionar o solucionar problemas de su configuración manualmente desde un editor de texto, además de introducir riesgos de seguridad asociados a la ejecución de código arbitrario durante la deserialización.

Implementación de escritura segura y respaldo

Para cumplir con el requerimiento de evitar la corrupción del archivo ante un cierre abrupto, se implementó un flujo transaccional de tres pasos:

Respaldo iterativo (.bak): Se lee el archivo original línea por línea (minimizando el uso de memoria caché para archivos grandes) y se escribe en un archivo con extensión .bak.

```
with open(CONFIG_FILE, "r", encoding="utf-8") as original, \
    open(BACKUP_FILE, "w", encoding="utf-8") as backup:
```

Archivo Temporal (.tmp): La nueva configuración se guarda en un archivo temporal.

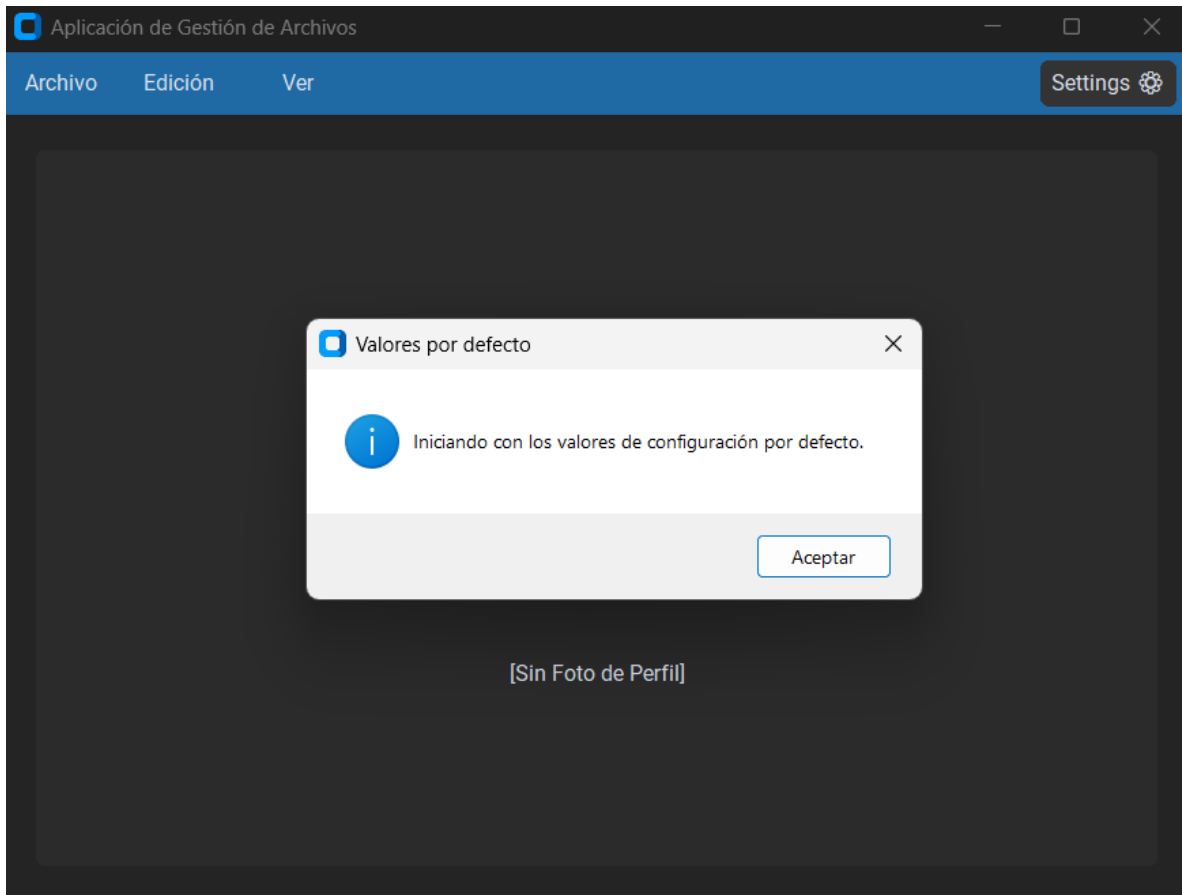
```
with open(TEMP_FILE, "w", encoding="utf-8") as temporal:
    json.dump(nueva_config, temporal, indent=4, ensure_ascii=False)
```

Reemplazo: Se utiliza la función a nivel de sistema operativo `os.replace()` para sobrescribir el archivo original.

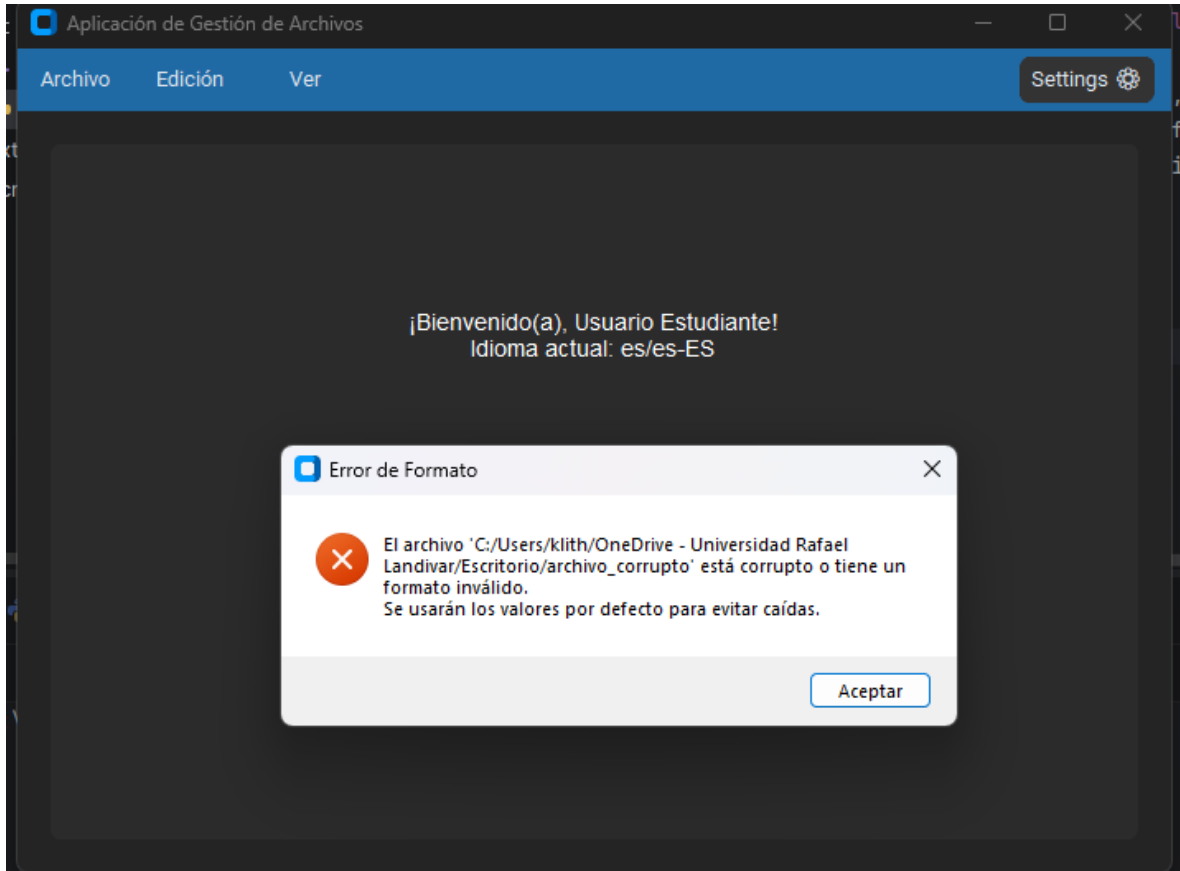
```
os.replace(TEMP_FILE, CONFIG_FILE)
return True
```

Manejo de errores de archivo:

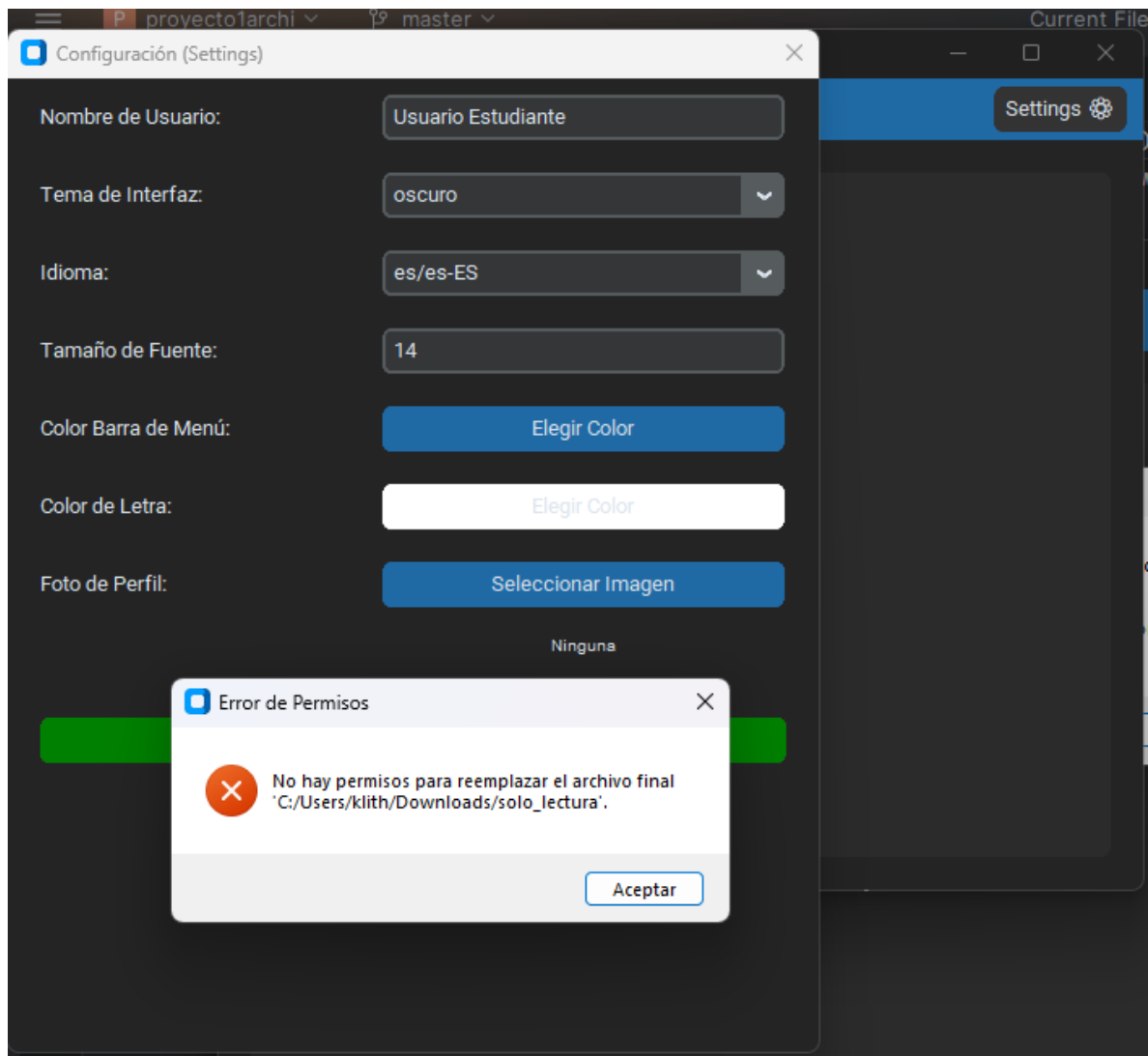
Archivo Ausente: Se detecta tempranamente utilizando `os.path.exists()`. Si no existe, se notifica al usuario y se retorna una copia profunda (`copy()`) de la configuración por defecto.



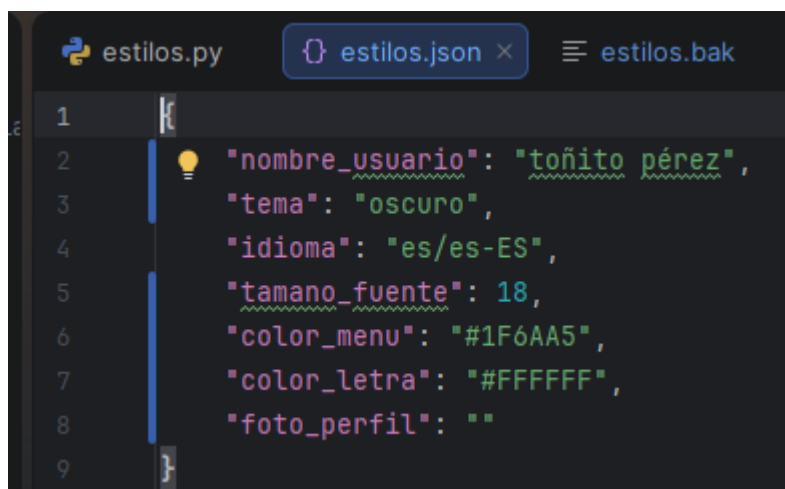
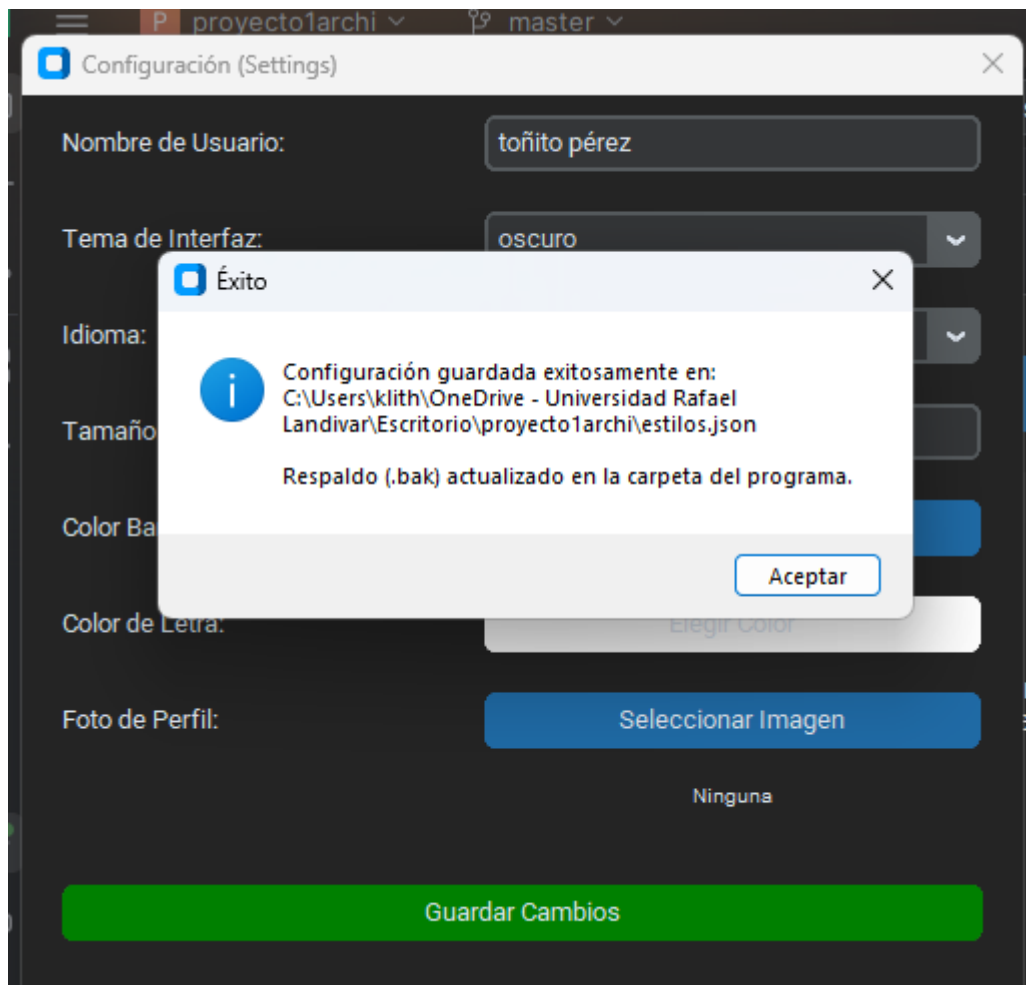
Archivo Corrupto: Si el archivo existe pero su contenido no es un JSON válido, el método `json.load()` lanza un `json.JSONDecodeError` el cual identifica el error que luego muestra una alerta advirtiendo la corrupción y se cargan los valores por defecto.



Falta de Permisos: Se envuelven las operaciones de `open()` y `os.replace()` dentro de bloques `try-except PermissionError` donde si el sistema operativo deniega la lectura o escritura (por ejemplo, si el archivo es de "Solo lectura"), se interrumpe la operación de forma segura y se notifica al usuario.



Evidencia del archivo generado y codificación UTF-8:



Funcionamiento de la aplicación:

